

MORLD - Chemical Molecule Generation Using LSTM and Reinforcement Learning

Abstract

This project explores generative modeling of chemical molecules through deep learning techniques. An LSTM network is trained to learn SMILES strings from a curated subset of the ZINC database. Reinforcement learning (RL) is subsequently employed to fine-tune the model toward generating molecules with optimized chemical properties. This approach facilitates the discovery of novel, valid, and property-optimized drug like molecules.

1. Problem Statement and Motivation

The pharmaceutical industry faces persistent challenges in discovering novel chemical compounds with desirable biological activity. Traditional discovery methods are often time-consuming, resource-intensive, and costly.

Motivation: This project aims to leverage machine learning specifically deep learning and reinforcement learning—to automate and enhance the molecular design process, providing innovative pathways to accelerate drug development.

2. Authors

TKSS Ashok , EE (b.tech), IIT Bombay

Gogineni Sumanth, EE (dual degree), IIT Bombay

Kotipalli Venkata Sriram, CSE (b.tech), IIIT VADODARA

3. Introduction

SMILES (Simplified Molecular Input Line Entry System) is a text-based representation of chemical structures. Long Short-Term Memory (LSTM) networks can be trained to predict and generate valid SMILES strings. Further fine-tuning using reinforcement learning (RL) allows the model to generate molecules that optimize certain properties, such as drug-likeness or the presence of specific chemical fragments.

4. End-to-End Project Workflow

The following is the step-by-step architecture of the project pipeline, highlighting how data flows from raw SMILES strings to molecule generation and evaluation.

Workflow Diagram:

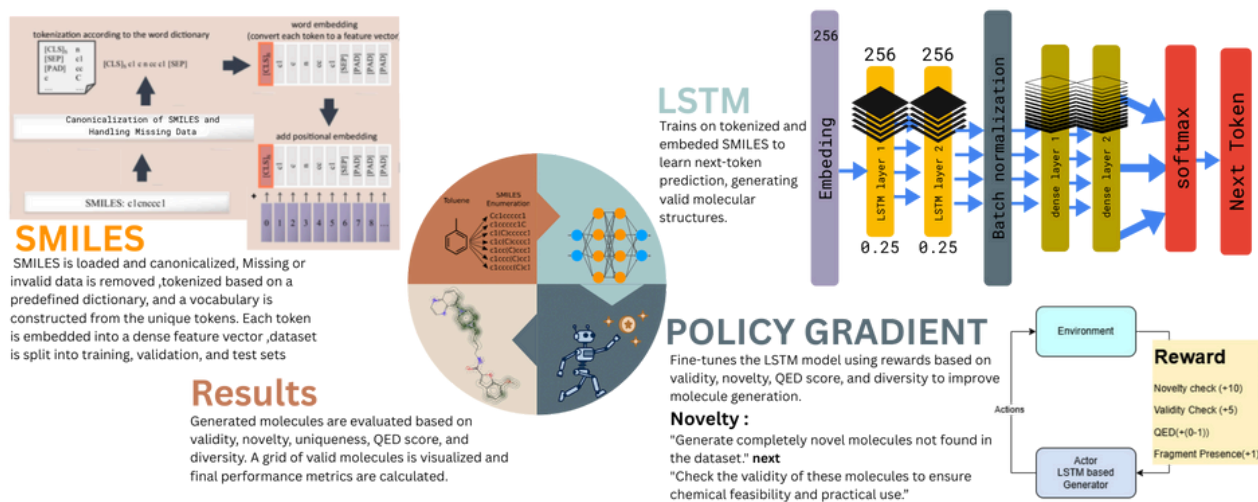


Figure 1: End-to-End Workflow for MORLD.

1. Raw Data Collection: Downloaded the entire ZINC database containing 1 crore (10 million) SMILES strings.
2. Preprocessing & Canonicalization: Canonicalized SMILES using RDKit and removed corrupted strings (preprocess.py).
3. Tokenization: Tokenized SMILES strings to create a vocabulary (tokenize smiles.py).
4. Dataset Creation: Created 10 datasets of 10 lakh SMILES each.
5. Local Token Mapping: Generated token-to-index mappings for each dataset.
6. Data Splitting: Split datasets into 70% training and 30% testing (split data tokens.py).
7. Fragment Augmentation: Replaced 30% of training molecules with molecules containing desired fragments during each epoch.
8. LSTM Model Training: Trained LSTM on SMILES (train model.py).
9. RL Fine-Tuning (REINFORCE): Fine-tuned with a custom reward (RLfinetune.py).
10. Molecule Generation: Generated SMILES using nucleus sampling (testRLLSTM.py).
11. Postprocessing & Evaluation: Analyzed Validity, Novelty, Diversity, QED, and Uniqueness.
12. Visualization: Created grid images of valid molecules

Novelty Enforcement: During RL fine-tuning, a high reward was given to novel molecules, ensuring the generation of unique structures.

Reward Components:

- Novelty Bonus (+10)
- Validity (+5)
- Fragment Matching (+1)
- QED (continuous reward)

4. Datasets

Source: ZINC Database
(<https://zinc.docking.org/>)

Original Data:

- 10 million SMILES strings.

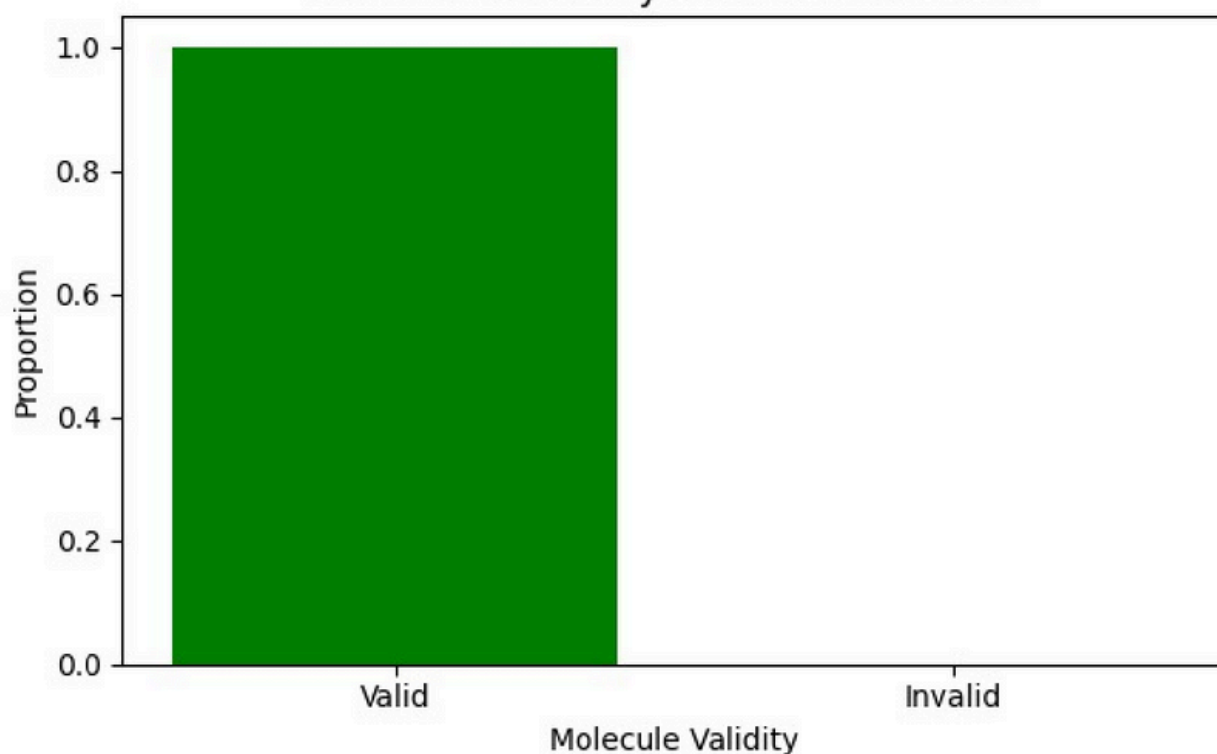
Preprocessed Data:

- Canonicalized SMILES strings.
- Global and local tokenization.
- 10 random datasets created.

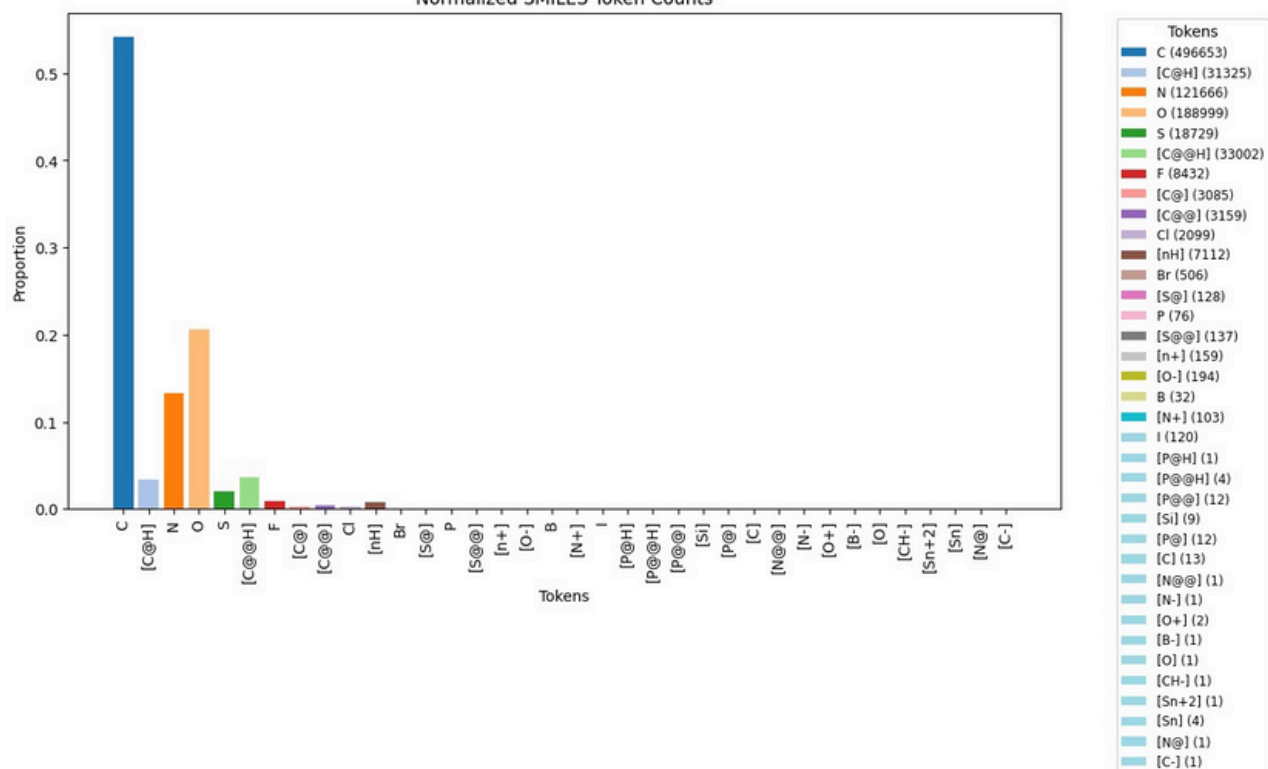
5. Features Extracted from Dataset

- Character-level tokens.
- Functional group detection (C=O, N-H, O-H).
- QED score.
- Metrics: Validity, Uniqueness, Novelty.

Normalized Validity of SMILES Molecules



Normalized SMILES Token Counts



6. Model Architecture

LSTM Generator:

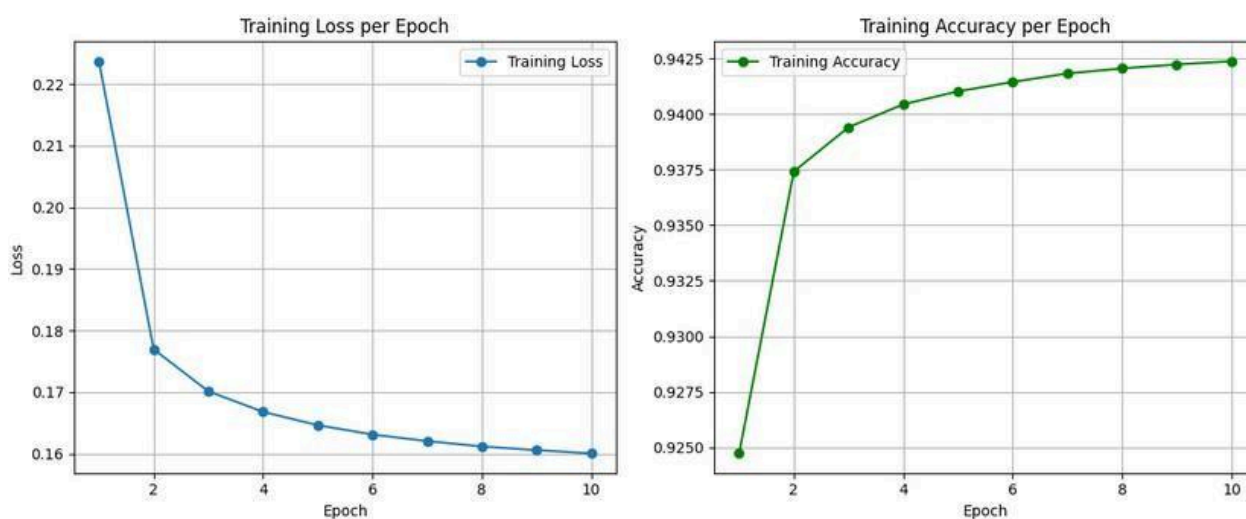
- Embedding Layer
- 2 LSTM Layers (hidden size 256)
- Dense Output Layer with Softmax

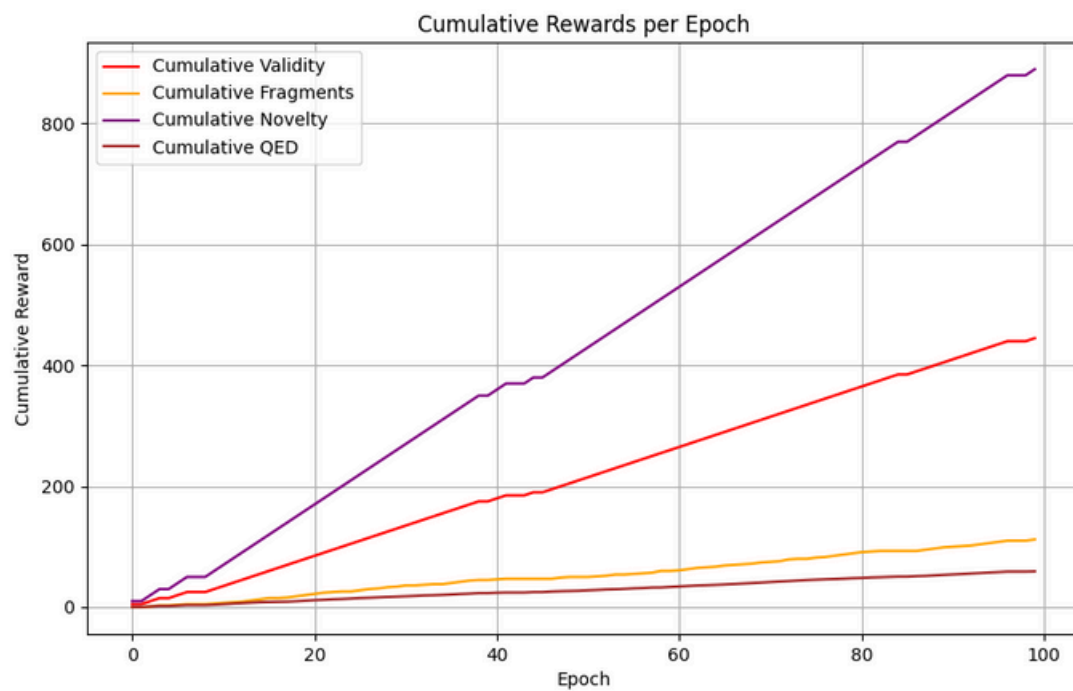
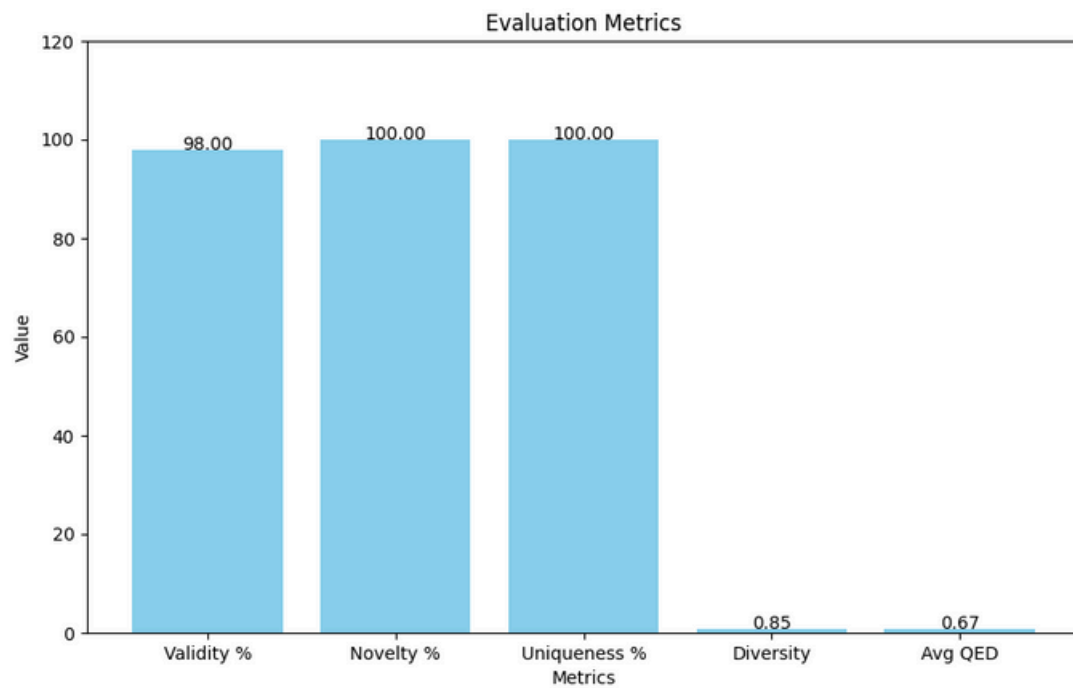
Reinforcement Learning Fine-Tuning:

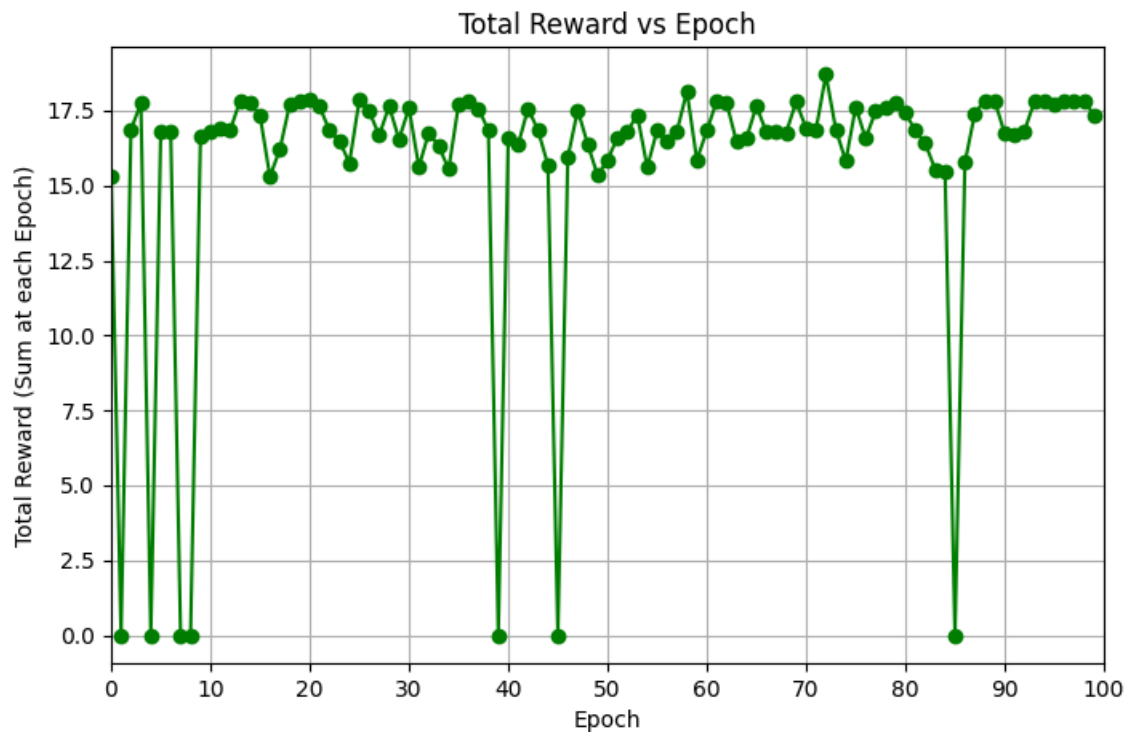
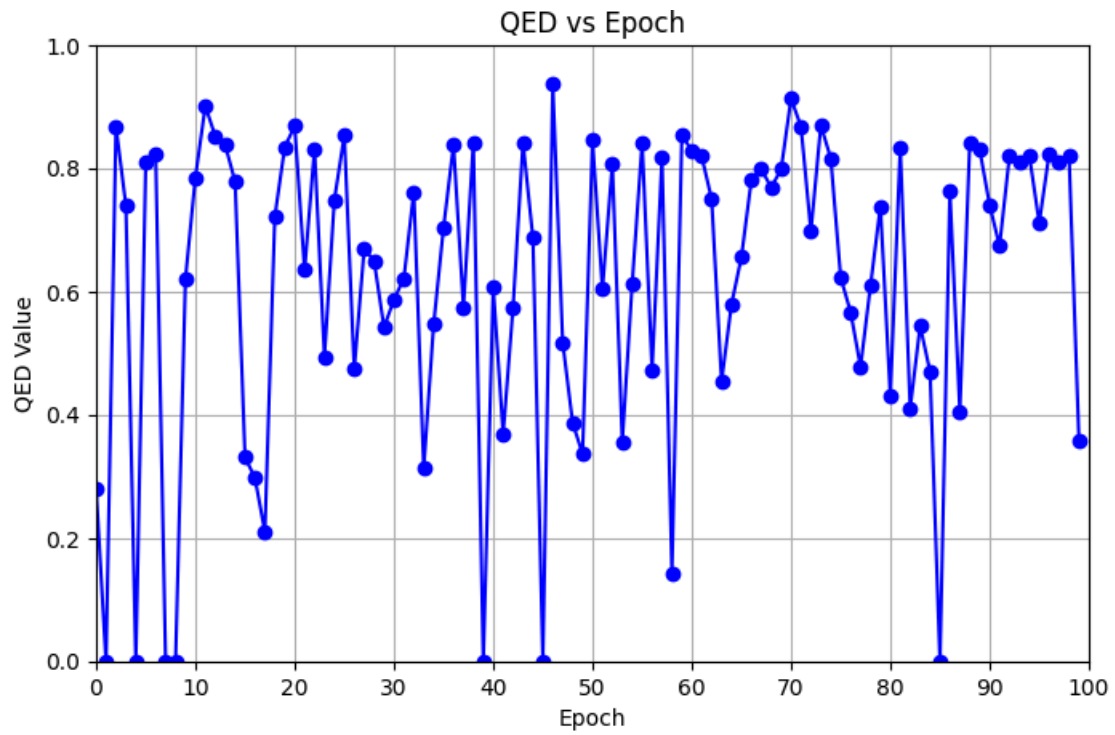
- Reward based on Validity, Novelty, Fragment Presence, and QED.
- Policy Gradient update (REINFORCE algorithm).

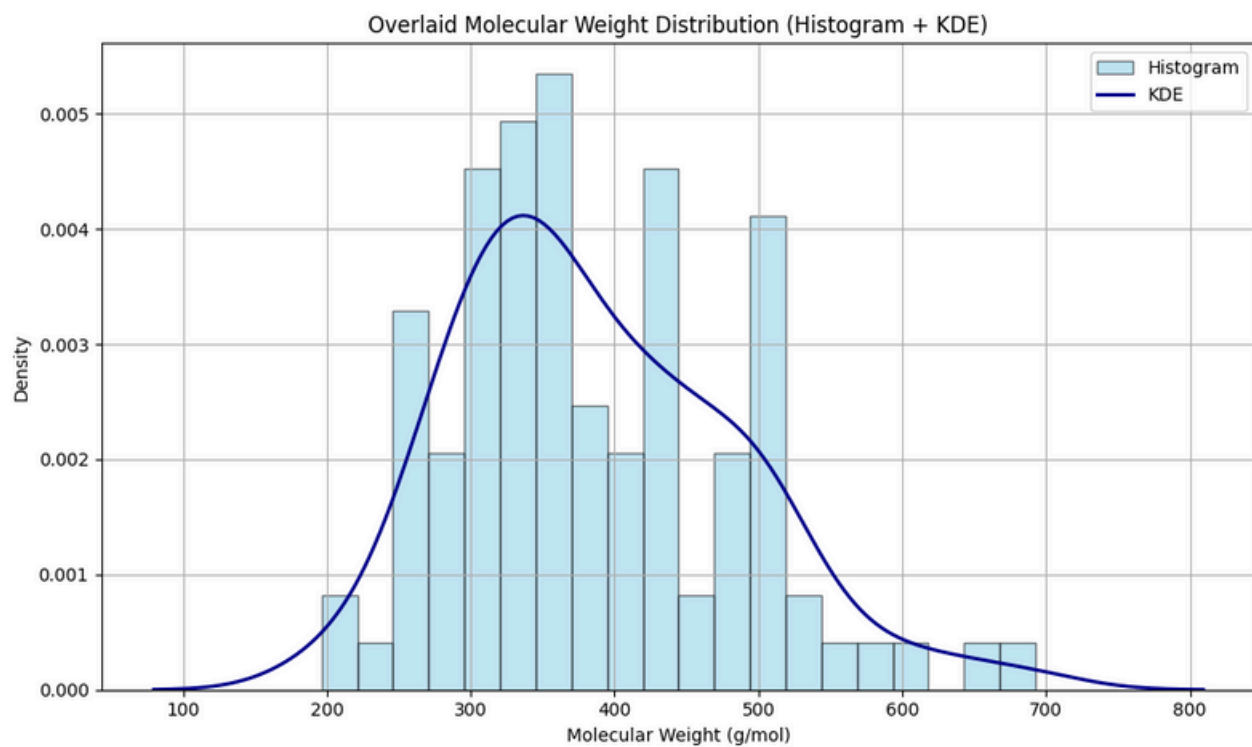
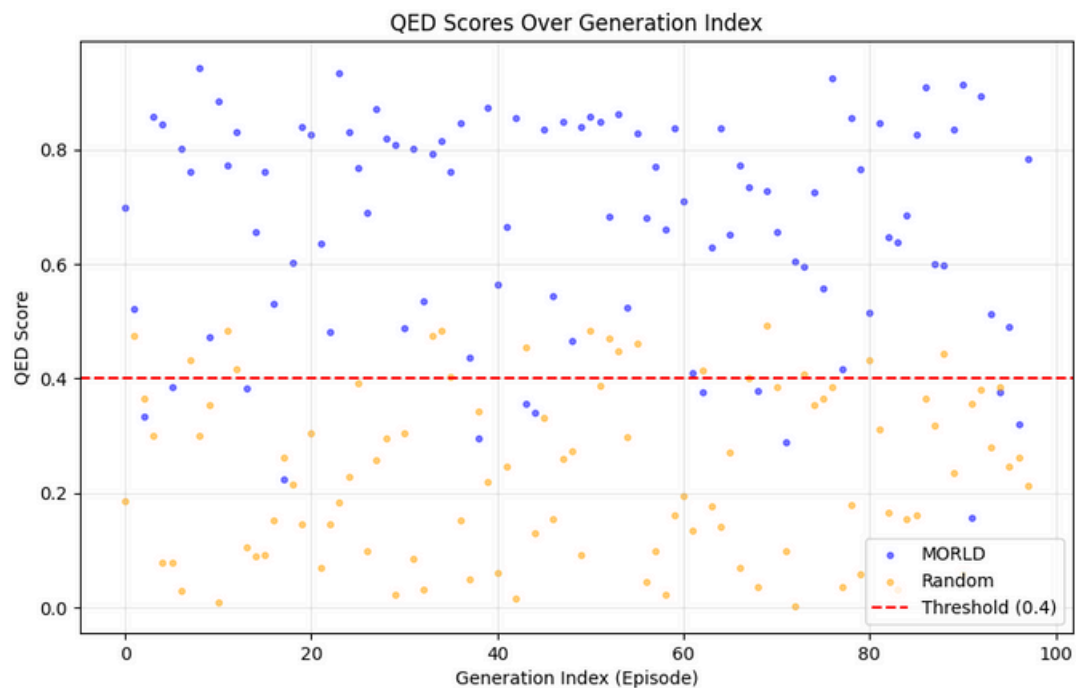
7. Results and Explanation

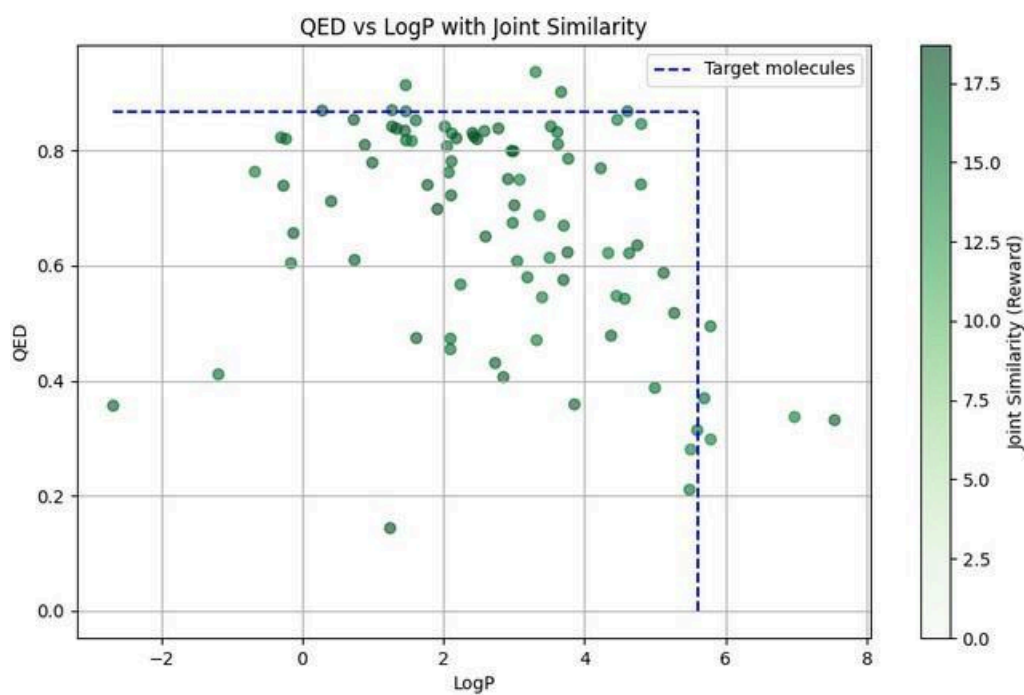
7.1 High syntactic validity in generated SMILES



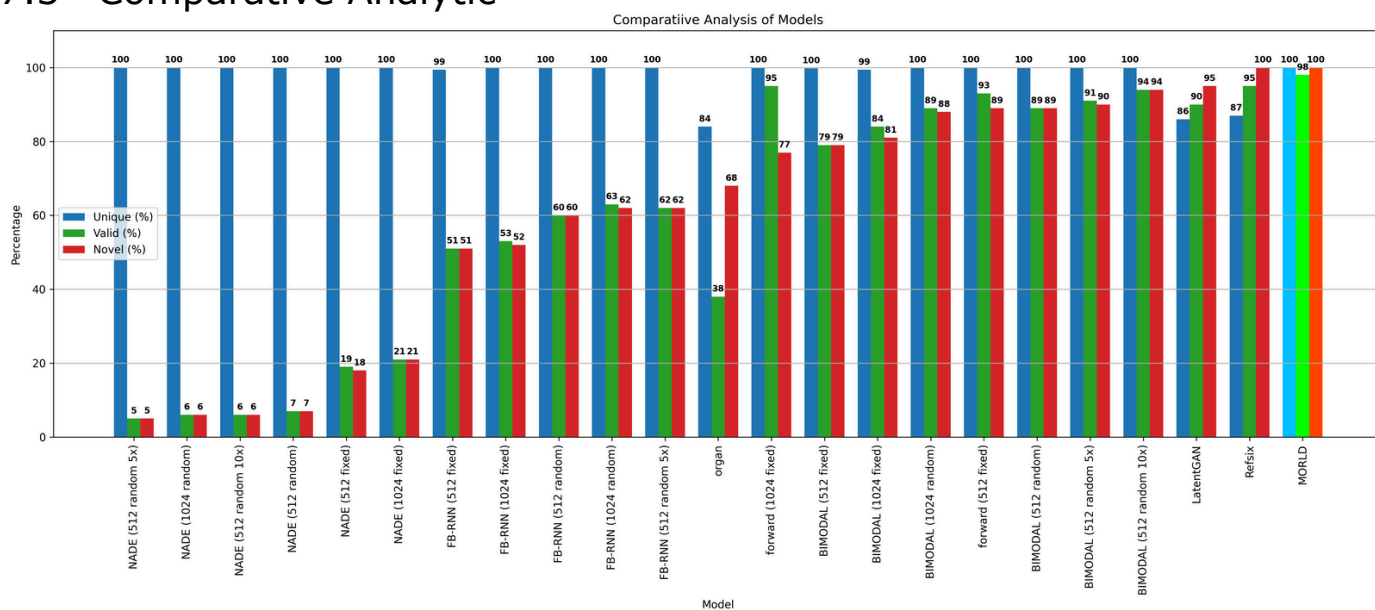




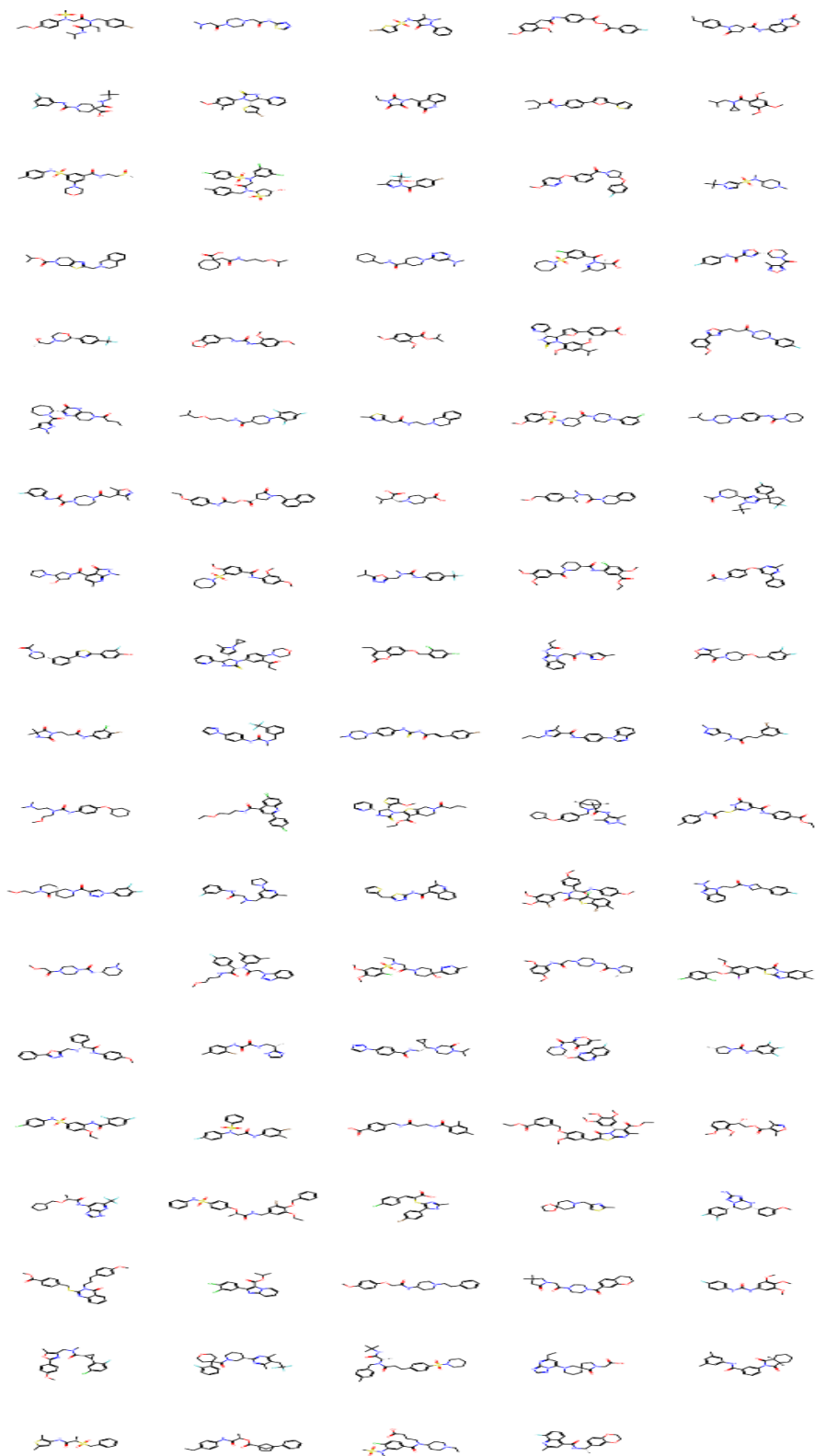




7.3 Comparative Analytic



8. Generated Molecules



9. Merits and Demerits

Merits:

- High validity rate.
- Reward customization possible.
- Accelerates drug discovery process.

De-merits:

- Computationally expensive.
- Requires domain-specific reward design.

10. Full Code and Execution Procedure

Code Structure:

- codes/preprocess.py - Preprocessing.
- codes/tokenize_smiles.py - Tokenization.
- codes/split_data_tokens.py - Splitting.
- codes/train_model.py - LSTM Training.
- codes/RLfinetune.py - Reinforcement Fine-Tuning.
- test.py - Testing.

Execution Steps:

- Install dependencies: `pip install rdkit-pypi torch numpy pandas`.
- Preprocess: `python codes/preprocess.py`.
- Tokenize: `python codes/tokenize smiles.py`.
- Split: `python codes/split_data tokens.py`.
- Train: `python codes/train_model.py`.
- Fine-tune: `python codes/RLfinetune.py`.
- Test: `python codes/testRLLSTM.py`.

11. Proper Documentation

All code is commented. Jupyter notebooks have markdowns explaining each stage. Outputs are saved.

12. Conclusion

Combining LSTM with reinforcement learning enables the generation of novel, valid molecules and accelerates drug discovery.

13. Future Scope

- Using Transformer architectures.
- Multi-objective optimization.
- Applying Mean Field Reinforcement Learning.

14. References

- Olivecrona, M., et al. (2017). Molecular de-novo design through deep reinforcement learning. Journal of Cheminformatics. <https://jcheminf.biomedcentral.com/articles/10.1186/s13321-017-0235-x>
- Arú's-Pous, J., et al. (2020). SMILES-based deep generative scaffold decorator.
- ZINC Database (<https://zinc.docking.org/>)
- RDKit Documentation (<https://www.rdkit.org>)
- Blaschke, T., et al. (2020). Memory-assisted reinforcement learning for diverse molecular de novo design.
- Grisoni, F., et al. (2020). Bidirectional molecule generation.
- Wang, Q., et al. (2023). Molecular generation strategy using A2C RL.
- Jeon, W., & Kim, D. (2020). Autonomous molecule generation using RL and docking.